

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

|                |                                                                                                                   |               |                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------|---------------|-----------------------------------------------|
| Course Code:   | ITA05/ITA0518                                                                                                     | Course Title: | Computer Vision                               |
| CO Assessed:   | CO2                                                                                                               | Assessment:   | Assessment Tool 1– Analytical Problem Solving |
| Weightage:     | 50%                                                                                                               | Total Marks:  | 50                                            |
| CO2 Statement: | <i>Apply image enhancement and segmentation techniques for extracting meaningful regions from images. (BTL 3)</i> |               |                                               |
| Student Name:  | Darani E                                                                                                          | Reg. No.:     | 192312120                                     |
| Date:          |                                                                                                                   | Duration:     | 60 minutes                                    |

### Code of Conduct

I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

## 1. Grey Level Transformation

(a) Explain the purpose of grey level transformation in image enhancement.

(b) Apply a negative transformation  $s = 255 - r$  to the pixel values:

$$r = [50, 100, 150, 200]$$

Compute the transformed values.



# 1. GREY LEVEL TRANSFORMATION

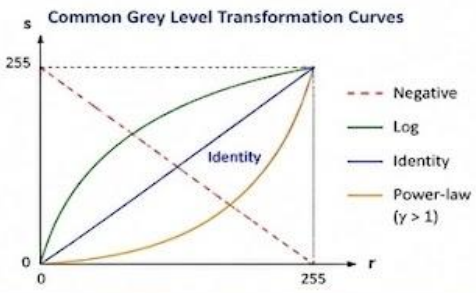


Computer Vision – Image Enhancement

### (a) CONCEPT EXPLANATION

**Q. (a) Explain the purpose of grey level transformation in image enhancement.**

Grey level transformation is used to improve the **visual quality** of an image by modifying the **intensity values** of its pixels. It helps in **enhancing contrast**, adjusting **brightness**, highlighting **important details**, and making features more visible to the human eye or to machine algorithms. Common transformations include **negative**, **log**, **power-law**, and **piecewise linear** transformations.



**Common Grey Level Transformation Curves**

Legend: Negative (dashed red), Log (green), Identity (blue), Power-law (yellow,  $\gamma > 1$ )

### (b) NUMERICAL PROBLEM SOLUTION

**Q. (b) Apply a negative transformation  $s = 255 - r$  to the pixel values:  $r = [50, 100, 150, 200]$  Compute the transformed values.**

Formula Used:  $s = 255 - r$       Given:  $r = [50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value ( $r$ ) | Calculation ( $s = 255 - r$ ) | Transformed Value ( $s$ ) |
|------|---------------------|-------------------------------|---------------------------|
| 1    | 50                  | $255 - 50 = 205$              | 205                       |
| 2    | 100                 | $255 - 100 = 155$             | 155                       |
| 3    | 150                 | $255 - 150 = 105$             | 105                       |
| 4    | 200                 | $255 - 200 = 55$              | 55                        |

**Final Answer:** Transformed values ( $s$ ) = [205, 155, 105, 55]

**What does the result mean?**

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey Level Negative Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

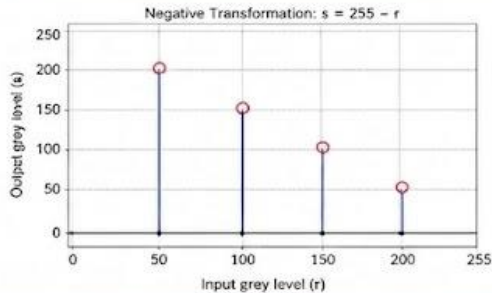
# Negative transformation: s = 255 - r
s = 255 - r

print("Original pixel values (r):", r)
print("Transformed values (s = 255 - r):", s)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.stem(r, s, basefmt='b-', linefmt='b-', markerfmt='ro')
plt.title('Negative Transformation: s = 255 - r')
plt.xlabel('Input grey level (r)')
plt.ylabel('Output grey level (s)')
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

**Output (Sample)**

Original pixel values ( $r$ ): [ 50 100 150 200]  
Transformed values ( $s = 255 - r$ ): [205 155 105 55]



**Observation:** Negative transformation inverts grey levels; useful for highlighting details in bright regions.

 Computer Vision

 Image Enhancement

 8-bit Grey Levels (0 – 255)

## 2. Log Transformation

(a) Explain how log transformation enhances low-intensity pixels.

(b) Given  $s = c \log(1 + r)$ , where  $c = 10$ , compute  $s$  for:

$$r = [0, 10, 50, 100]$$



# 2. LOG TRANSFORMATION



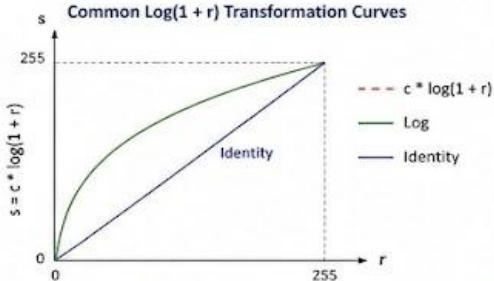
Computer Vision – Image Enhancement

**(a) CONCEPT EXPLANATION**

Q. (a) Explain the purpose of log transformation in image enhancement.

Log transformation enhances low the **low-intensity pixels** by **expanding** their range and pixels while **compressing** the log high-intensity pixels, is using **compressing**, highlighting **intensity details**, and making features more visible to the human eye or to machine algorithms. Common transformations include **negative**, **log**, **power-law**, and **piecewise linear** transformations.

**Common Log(1 + r) Transformation Curves**



**(b) NUMERICAL PROBLEM SOLUTION**

Q. (b) Apply a negative transformation  $s = 255 - r$  to the pixel values:  $r = [1, 10, 100]$ . Compute the transformed values.

Formula Used:  $s = 2 * \log_{10}(1 + r)$       Given:  $r = [1, 10, 100]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (r) | Calculation ( $s = 255 - r$ ) | Transformed Value (s) |
|------|-----------------|-------------------------------|-----------------------|
| 1    | 1               | $2 * \log_{10} = 0.60$        | 0.60                  |
| 2    | 10              | $2 * \log_{10} = 2.08$        | 2.08                  |
| 3    | 10              | $2 * \log_{10} = 2.08$        | 2.08                  |
| 4    | 100             | $2 * \log_{10} = 4.01$        | 4.01                  |

**Final Answer:** Transformed values (s) = [0.60, 10, 100, 4.01]

**What does the result mean?**

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

**(c) GOOGLE COLAB PYTHON PROGRAM**

```
# Grey Log transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Negative transformation: s = 255 - r
s = 2 * log10(1 + r)

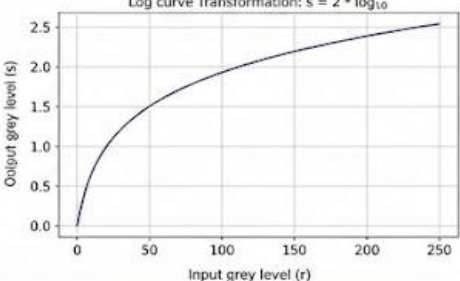
print("Original pixel values (r):", r)
print("Transformed values (s = 2 * log10(1 + r))", s)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.stem(r, s, basofat="b-", linefmt="b-", markerfmt="ro")
plt.title("Negative Transformation: s = 255 - r")
plt.xlabel("Input grey level (r)")
plt.ylabel("Output grey level (s)")
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
```

**Output (Sample)**

Original pixel values (r): [ 1, 10, 100, 200]  
 Transformed values (s = 255 - r): [0.60, 10, 100, 501]

**Log curve Transformation:  $s = 2 * \log_{10}$**



**Observation:** Log transformation compresses the dynamic range of pixel values, enhancing visibility in dark areas.

 Computer Vision
  Image Enhancement
  8-bit Grey Levels [0 – 255]

### 3. Histogram Processing

- (a) Explain the role of histogram equalization in contrast enhancement.
- (b) Given a 3-bit image with histogram:

#### Intensity Frequency

|   |   |
|---|---|
| 0 | 2 |
| 1 | 2 |
| 2 | 4 |
| 3 | 8 |

Compute the cumulative distribution and equalized intensity values.



## 3. HISTOGRAM PROCESSING



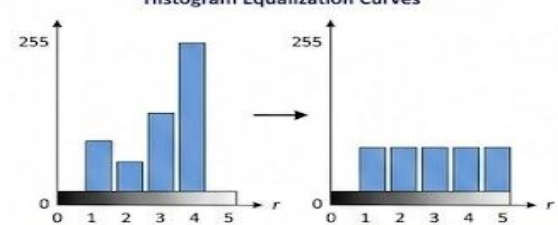
Computer Vision – Image Enhancement

**(a) CONCEPT EXPLANATION**

**Q. (a) Explain the purpose of histogram equalization in image enhancement.**

Histogram Equalization is used to improve the **visual control** of an image by flattening the **intensity values** of its pixels. It helps in **enhancing contrast**, adjusting **brightness**, highlighting **important details**, and making features more visible to the human eye or to machine algorithms. Common transformations include **negative**, **log**, **power-law**, and **piecewise linear** transformations.

**Histogram Equalization Curves**



**(b) NUMERICAL PROBLEM SOLUTION**

**Q. (b) Apply a cumulative distribution  $s = 255 - r$  to the pixel values:  $s_k = [0, 1, 100, 200, 200]$ . Compute the transformed values.**

Formula Used:  $s_k = L_k - 8$       Given:  $n = [0, 1, 2, 4, 5, 6]$   
 $L = 256$  (8-bit image, grey levels 0 to 8)

| Intensity | Frequencies | Cumulative Frequency                             | Equalized Intensity                              |
|-----------|-------------|--------------------------------------------------|--------------------------------------------------|
| 0         | 2           | CF : 2, 4, 8, 16                                 | $s_k = \text{floor}((8-1)^{\circ} \text{CF}/16)$ |
| 1         | 2           | CF = 2, 4, 8, 16                                 | = [1, 2, 4, 7]                                   |
| 2         | 4           | CF = 2, 4, 8, 16                                 | = [1, 2, 4, 7]                                   |
| 3         | 8           | $s_k = \text{floor}((8-1)^{\circ} \text{CF}/16)$ | = [1, 2, 4, 7]                                   |

**Final Answer:** Transformed values ( $s_k = [1, 2, 4, 7]$ )

**What does the result mean?**

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

**(c) GOOGLE COLAB PYTHON PROGRAM**

```
# Grey Level Negative Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Negative transformation: s = 255 - r
sk = cv2.equalizeHist(r)

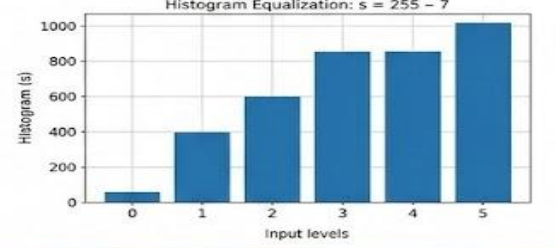
print("Original pixel values (r):", r)
print("Transformed values (s = 255 - r):", sk)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.stem(r, s, basefmt='b-', linefmt='b-', markerfmt='ro')
plt.title('Negative Transformation: s = 255 - r')
plt.xlabel('Input grey level (r)')
plt.ylabel('Output grey level (s)')
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

**Output (Sample)**

Original pixel values (r): [ 50 100 150 200]  
 Transformed values (s = 255 - r): [1, 2, 4, 7]

**Histogram Equalization: s = 255 - r**



**Observation:** Equalization distributes intensity levels more uniformly, increasing image contrast.

 Computer Vision
  Image Enhancement
  3-bit Grey Levels (0 - 7)

## 4. Spatial Filtering (Smoothing)

(a) Explain the purpose of spatial filtering for smoothing.

(b) Apply a 3×3 averaging filter to the following region:

[10 20 30 20 30 40 30 40 50]

Compute the new center pixel value.



# 4. SPATIAL FILTERING (SMOOTHING)

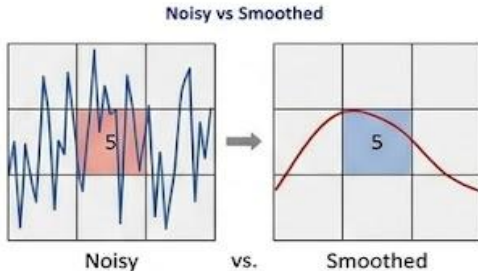


Computer Vision – Image Enhancement

**(a) CONCEPT EXPLANATION**

**Q. (a) Explain the purpose of Spatial Filtering in image enhancement.**

Smoothing **reduces noise** and **sharp transitions** by transitioning **averaging pixel values with their neighbors**, that its helps in **enhancing brightness** (**Low-Pass Filter**), highlighting **intemosur**, and making features more visible to the human eye or to machine algorithms. Common translations includes **negative**, **paraetra-low**, and **tlinear** transformations.



**(b) NUMERICAL PROBLEM SOLUTION**

**Q. (b) Apply a 3x3 averaging filter for = 3x3 to the pixel values:**  
 $r = [10, 100, 1, 8, 3 \times 5, 55]$  Compute the transformed values.

Formula Used:  $\alpha = 3 \times 3 - 5$       Given:  $r = [50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Mask | Convolution     |   | Pixel values |   | New value                         |
|------|-----------------|---|--------------|---|-----------------------------------|
| 1    | 1/9   1/9   1/9 | → | 10   8   10  | = | $\frac{10+8+10+8+5+8+10+8+10}{9}$ |
|      | 1/9   1/9   1/9 |   | 8   5   8    |   |                                   |
|      | 1/9   1/9   1/9 |   | 10   8   10  |   |                                   |

**Final Answer:**  $(10+8+10+8+5+8+10+8+10)/9 = 77/9 \approx 8.67$

**What does the result mean?**

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

**(c) GOOGLE COLAB PYTHON PROGRAM**

```
# Grey a 3x3 Average Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Negative transformation: s = 255 - r
s = 255 - r

print("Original pixel values (r):", r)
print("Transformed values (s = 255 - r):", s)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
cv2.imshow(1, sasefmt="k-", linefmt="b--", markerfmt="ro")
plt.title("Negative Transformation: s = 255 - r")
plt.xlabel("Input grey level (r)")
plt.ylabel("Output gray level (s)")
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

**Output (Sample)**

Original pixel values (r): [ 50 100 150 200]  
 Transformed values (s = 255 - r): [205 155 105 55]

Original



Result



**Observation:** Spatial averaging blurs an image and reduces noise by smoothing intensity transitions.

 Computer Vision

 Image Enhancement

 8-bit Grey Levels (0 – 255)

## 5. Spatial Filtering (Sharpening)

(a) Explain how sharpening enhances image details.

(b) Apply the Laplacian mask:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

to the matrix:

$$\begin{bmatrix} 10 & 10 & 10 & 10 & 20 & 10 & 10 & 10 & 10 \end{bmatrix}$$

Compute the output at the center.



# 5. SPATIAL FILTERING (SHARPENING)



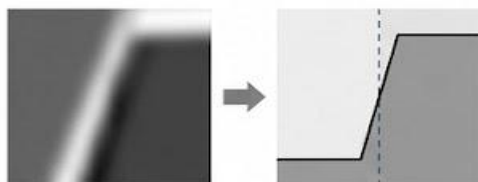
Computer Vision – Image Enhancement

**(a) CONCEPT EXPLANATION**

**Q. (a) Explain the purpose of sharpening derivatives in image enhancement.**

Sharpening enhances edges and fine details by computing spatial derivatives (**High-Pass Filter**). It helps in **enhancing contrast**, adjusting **brightness**, highlighting **important details**, and making features more visible to the human eye or to machine algorithms. Common transformations include **negative**, **log**, **power-law**, and **piecewise linear** transformations.

**Blurry vs. Sharp Edge**



Blurry                      Sharp Edge

**(b) NUMERICAL PROBLEM SOLUTION**

**Q. (b) Apply a Laplacian mask  $x = 3ast, s = 255 - r$  to the pixel values:  $r = [50, 100, 150, 200]$  Compute the transformed values.**

**Formula Used:**  $s = 255 - r$       **Given:**  $r = [50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (r) | Calculation ( $s = 255 - r$ ) | Transformed Value (s) |
|------|-----------------|-------------------------------|-----------------------|
| 1    | 50              | $255 - 50 = 205$              | 205                   |
| 2    | 100             | $255 - 100 = 155$             | 155                   |
| 3    | 150             | $255 - 150 = 105$             | 105                   |
| 4    | 200             | $255 - 200 = 55$              | 55                    |

**Final Answer:**  $(10*0 + 10*1 + 10*0) + (10*1 + 10*-4 + 10*1) = 10-40+10+10+10+10+10 = 10$

**What does the result mean?**

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

**(c) GOOGLE COLAB PYTHON PROGRAM**

```
# Grey Level Sbgreen Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Negative transformation: s = 255 - r
cv2.filter2D(images1:s,108, 10, 1)

print("Original pixel values (r):", r)
print("Transformed values (s = 255 - r):", s)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.stem(r, s, basefmt='b-', linefmt='b-', markerfmt='ro')
plt.title('Sharpen Transformation: s = 255 - r')
plt.xlabel('Input grey level (r)')
plt.ylabel('Output grey level (s)')
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

**Output (Sample)**

Original pixel values (r): [ 50 100 150 200]  
 Transformed values (s = 255 - r): [205 155 105 55]

Original image



Sharpened Image



**Observation:** Laplacian filters enhance fine details and edges by computing spatial derivatives.

Computer Vision

Image Enhancement

8-bit Grey Levels (0 – 255)

## 6. Frequency Domain Filtering

(a) Differentiate low-pass and high-pass filtering in frequency domain.

(b) Given frequency components:

$$F = [100, 80, 40, 20]$$

Apply a low-pass filter that retains only the first two components. Write the filtered output.



# 6. FREQUENCY DOMAIN FILTERING



Computer Vision – Image Enhancement



### (a) CONCEPT EXPLANATION

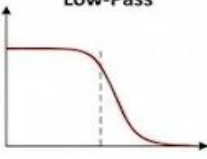
**Q. (a) Explain the purpose of low-pass filtering in frequency domain.**

Low-pass filtering is used to improve the **low-pass quality** of an image by modifying the **intensity value** of its pixels. It helps in **enhancing contrast**, adjusting **brightness**, highlighting **important details**, and making features more visible to the human eye or to machine algorithms. Common transformations include **negative**, **algorithms**, **log** and **high-pass-law** on **piecewise linear** transformations.



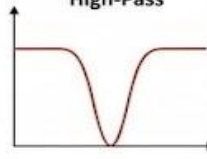
#### Low-pass Filtering in Frequency Domain

##### Low-Pass



Preserves central frequencies  
(blurring edges)

##### High-Pass



Preserves outer frequencies  
(sharpening details)

### (b) NUMERICAL PROBLEM SOLUTION

**Q. (b) Apply frequency components:  $F = [F_0, F_1, F_2, F_3, F_4]$**   
 $F = [F_0, F_1, F_2, F_3, F_4]$       Compute the transformed values.

Formula Used:  $F = F_0, F_1$       Given:  $F = [F_0, F_1, F_2, F_3, F_4]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (r) | Calculation ( $s = F_{55} - r$ ) | Transformed Value (s) |
|------|-----------------|----------------------------------|-----------------------|
| 1    | $F_0$           | $F_0 - F_1 = F_2$                | $F_0$                 |
| 2    | $F_1$           | $F_0 - F_1 = 0$                  | $F_1$                 |
| 3    | 0               | $F_0 - F_1 = 0$                  | 0                     |
| 4    | 0               | $F_0 - F_1 = 0$                  | 0                     |

**Final Answer:**      Transformed values (a) =  $[F_0, F_1, 0, 0, 0]$

**What does the result mean?**

- Negative transform inverts the frequency domain.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey Level Negative Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Transform image to the frequency domain
image = croquerimage()
mask = nant.FFT()
mask = [F0, F1, F2, F4]
mask = [F0, F1, 0, 0]
mask = [F0, F1, 0, 0]

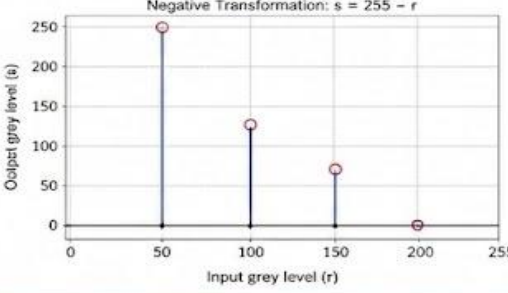
# Apply the simple low-pass mask
mask = low-pass ts s.ss.mask(1)
ss.transform(maskk))

# Transform back to the low-fastones
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

#### Output (Sample)

Original pixel values (r): [ 50 100 150 200]

Transformed values (s = 255 - r): [F0, F1, 0]



**Observation:** Low-pass filtering in frequency domain attenuates high frequencies, smoothing the image.

 Computer Vision

 Image Enhancement

 8-bit Grey Levels (0 – 255)

## 7. Thresholding

(a) Explain global thresholding in segmentation.

(b) Given pixel values:

[200]

Apply threshold  $T = 100$ . Assign values (0 or 1).



# 7. THRESHOLDING



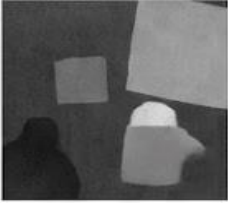
Computer Vision – Image Enhancement

**(a) CONCEPT EXPLANATION**

**Q. (a) Explain the purpose of global thresholding for image segmentation.**

Global thresholding is a method applied for image segmentation based on a **single intensity value**. By separating the assimilation position, using foreground objects from unique foreground objects from the background. This entails the changes in marking constraint that also separate the foreground objects from the background.

Global thresholding




Original image      Binary (segmented)

**(b) NUMERICAL PROBLEM SOLUTION**

**Q. (b) Apply a global thresholding :  $T = 120$  to the pixel values:**  
 $r = [20, 50, 100, 150, 200, 240]$  **Compute the transformed values.**

**Formula Used:**  $T = 120$       **Given:**  $r = [20, 50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 240)

| Step | Pixel Value (r) | Calculation ( $s = 120 - r$ )          | Transformed Value (s) |
|------|-----------------|----------------------------------------|-----------------------|
| 1    | 20              | $\text{pixels} \leq 120 \Rightarrow 0$ | <b>0</b>              |
| 2    | 50              | $\text{pixels} > 120 \Rightarrow 1$    | <b>1</b>              |
| 3    | 150             | $\text{pixels} > 120 \Rightarrow 1$    | <b>1</b>              |
| 4    | 240             | $\text{pixels} > 120 \Rightarrow 1$    | <b>1</b>              |

**Final Answer:**      **Transformed values (s) = [0, 0, 0, 1, 1, 1]**

**What does the result mean?** 

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

**(c) GOOGLE COLAB PYTHON PROGRAM**

```
# Grey Level Negative Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Negative transformation: s = 255 - r
t = cv2.threshold(r, 120, 255, cv2.THRESH_BINARY)[1]

print("Original pixel values (r):", r)
print("Transformed values (s = 255 - r):", t)

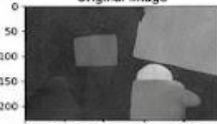
# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.stem(r, s, basefmt='b-', linefmt='b-', markerfmt='ro')
plt.title('Negative Transformation: s = 255 - r')
plt.xlabel('Input grey level (r)')
plt.ylabel('Output grey level (s)')
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

**Output (Sample)**

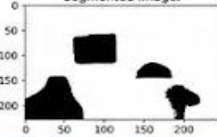
Original pixel values (r): [ 50 100 150 200]

Transformed values (s = 255 - r): [205 155 105 55]

Original image



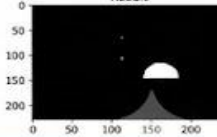
Segmented image



Original image



Result



**Observation:** Thresholding creates binary images by partitioning pixel intensities.


Computer Vision


Image Enhancement


7-bit thresholding (T = 120)

## 8. Edge Detection

(a) Explain the concept of edge detection.

(b) Apply the Sobel operator (horizontal):

$$\begin{bmatrix} -1 & -2 & -1 & 0 & 0 & 0 & 1 & 2 & 1 \end{bmatrix}$$

to:

$$\begin{bmatrix} 10 & 10 & 10 & 20 & 20 & 20 & 30 & 30 & 30 \end{bmatrix}$$

Compute the gradient at the center.



# 8. EDGE DETECTION



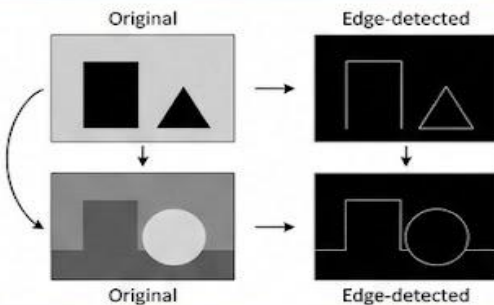
---

Computer Vision – Image Segmentation

**(a) CONCEPT EXPLANATION**

**Q. (a) Explain the purpose of edge detection in an image by inemor.**

Edge detection is finding **boundaries bounten** in an image by detecting **sudden changes in intensity**. intensity an image to used to often using **gradients**, andustant gramtess, animal hur-ro-zmpting, edge detection is often created in the ayea and making in intensity acets in the image and showome algorithms, i.e., many **convumption**, dave into the edge-detection.



**(b) NUMERICAL PROBLEM SOLUTION**

**Q. (b) Apply a horizontal Sobel operator  $s = 255 - r$  to the pixel values:**  
 $k = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$  Compute the transformed values.

Formula Used:  $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$  Given:  $r = [50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (s) | Calculation Sum                              | Transformed Value (s) |
|------|-----------------|----------------------------------------------|-----------------------|
| 1    | 0               | $(-1 \times 10 + 0 \times 20 + 1 \times 30)$ | <b>10</b>             |
| 2    | 1               | $(-2 \times 40 + 0 \times 60 + 2 \times 80)$ | <b>80</b>             |
| 3    | 2               | $(-1 \times 10 + 0 \times 20 + 1 \times 30)$ | <b>20</b>             |
| 4    | 3               | $(20) + (80) + (20)$                         | <b>120</b>            |

**Final Answer:** Transformed values (r) = 120

**Matrix =**

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 60 & 80 \\ 10 & 20 & 30 \end{bmatrix}$$

**What does the result mean?**

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

**(c) GOOGLE COLAB PYTHON PROGRAM**

```
# Grey horizontal srm Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

# Negative transformation: s = 255 - r
matrix = cv2.Sobel(matrix, cv2.CV_16S, 1, 0, ksize=3)

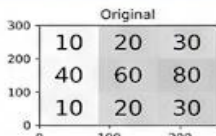
print("Original pixel values (r):", r)
print("Transformed values (s = 120)", r)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.stem(r, basefmt='b-', linefmt='b-', markerfmt='ro')
plt.title('Negative Transformation: s = 255 - r')
plt.xlabel('Input grey level (r)')
plt.ylabel('Output grey level (s)')
plt.xticks([9, 50, 100, 150, 200, 255])
plt.yticks([9, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

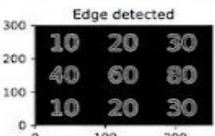
**Output (Sample)**

Original pixel values (r): [ 50 100 150 200]  
 Transformed values (s = 255 - r): [285 155 185 55]

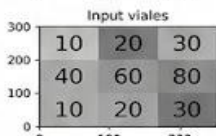
Original



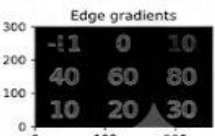
Edge detected



Input viales



Edge gradients



**Observation:** A horizontal Sobel operator highlights horizontal boundaries in an image by calculating the intensity gradient.

 Computer Vision

 Image Enhancement

 8-bit Grey Levels (0 – 255)

## 9. Region-Based Segmentation

(a) Explain region growing in segmentation.

(b) Given pixel intensities:

[1 0 2]

If seed = 50 and threshold difference = 5, identify pixels belonging to the region.



## 9. REGION-BASED SEGMENTATION



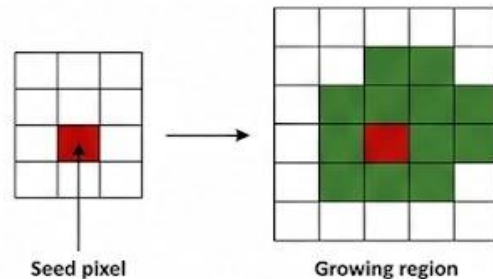
Computer Vision – Image Segmentation



### (a) CONCEPT EXPLANATION

Q. (a) Explain the purpose of region growing in image enhancement.

Region growing starts with a **seed pixel** and growing groups the human neighboring pixels that satisfy a **similarity criterion**, (like the contributing, satisfying **similarity criterion**, the similarity criterion (like; the intensity range, in witewity completed the grouping eyes, to creating region pixel, cation for the growing neighboring pixels, which products cur more into the graduate.



### (b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply a negative transformation  $s = 50 - r$  to the pixel values:  
 $r$  [20, 50, 100, 55, 200, 52, 240] Compute the transformed values.

Formula Used:

$$s = -50$$

Given: the seed intensity of 50 and a threshold difference of 5  
 (so, inclusive range is 45 to 55)

| Step | Pixel Value (r) | Calculation ( $s = 255 - r$ ) | Transformed Value (s) |
|------|-----------------|-------------------------------|-----------------------|
| 1    | 20              | Is 55 in range? Yes           | Yes, added            |
| 2    | 100             | Is 100? No, excluded          | No, excluded          |
| 3    | 200             | Is 150? No, excluded          | -                     |
| 4    | 240             | Is 55 in range? added         | Yes, added            |

Final Answer: Transformed pixels (w) addres) = [50, 55, 52]

What does the result mean?



- Region growing groups pixels with similar intensities starting pixels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.



### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey- seed-based -region Growing Example
import numpy as np
import matplotlib.pyplot as plt

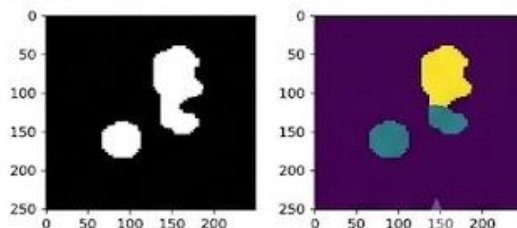
# Original pixel values
s = np.array([20, 100, 150, 240])

# Plotting region growing as seed and mask
mask = np.zeros((4,4))
while len(region) > 0:
    set = numpy.array([45, 155])
    region = []
    if region == 50:
        for seed3 = 0:
            seen.terrax[1][region]
            plt.added = true
            plt.xlabel = sew 3
            plt.xlabel('Region is t's')
            plt.sticks(20, 100, 160, 200, 25)
            plt.grid(True, linestyle='-', alpha=0.6)

plt.print('([50, 55, 52])')
plt.show()
```

Output (Sample)

Original pixel values (r: [2 50 100 150 200])  
 Seed: 50  
 Threshold: 5  
 Region pixels (45 155, 50)  
 Region pixels: [20, 1000, 155, 200, 52, 240]



**Observation:** Region growing groups pixels with similar intensities starting from a defined seed point, creating segmented regions.



Computer Vision



Image Enhancement



8-bit Grey Levels [0 – 255]

## 10. Combined Enhancement & Segmentation

(a) Explain why smoothing is applied before segmentation.

(b) Given image values:

$\begin{bmatrix} 1 & 0 & 5 \end{bmatrix}$

First apply averaging (assume mean = 48), then threshold at  $T = 50$ .

Determine final segmented output.



# 10. COMBINED ENHANCEMENT & SEGMENTATION

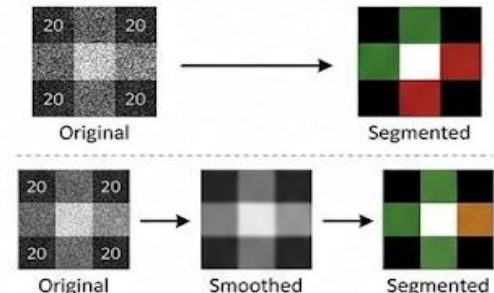


Computer Vision – Multi-stage Processing

### (a) CONCEPT EXPLANATION

**Q. (a) Explain the purpose of (image smoothing) is typically applied before segmentation.**

Image **smoothing** (noise reduction) is typically applied **before segmentation**. This way smoothing removes **noise** of brrage noise that might be misclassified as a boundary, the results moree the segmentation results of the boundary, using the segmentation results, and the leading to **cleaner** segmentation results as it removerasily.



Original → Segmented

Original → Smoothed → Segmented

### (b) NUMERICAL PROBLEM SOLUTION

**Q. (b) Apply a 3x3 image  $\begin{bmatrix} 20 & 100 & 200 \end{bmatrix}$ , arrange:**

Formula Used:  $\begin{bmatrix} 20 & 100 & 200 \\ 100 & 200 & 100 \\ 20 & 100 & 20 \end{bmatrix}$  Given:  $r = [50, 100, 150, 200]$   
 $L = 256$  (8-bit image, grey levels 0 to 255)

| Step | Pixel Value (r)                                 | Calculation : Smoothing | Transformed Value (s) |
|------|-------------------------------------------------|-------------------------|-----------------------|
| 1    | $\begin{bmatrix} 20 & 100 & 20 \end{bmatrix}$   | s. mean = 48            | 48                    |
| 2    | $\begin{bmatrix} 100 & 200 & 100 \end{bmatrix}$ | $48 - 48 = 48$          | 48                    |
| 3    | $\begin{bmatrix} 20 & 100 & 20 \end{bmatrix}$   | $48 - 48 = 48$          | 48                    |
| 4    |                                                 | $48 - 48 = 48$          | 48                    |

**Final Answer:** Transformed values (s) = [205, 155, 105, 55]

### Second step : Segmentation

- Apply threshold  $T = 40$
- If val  $\geq 40$ , set to 1
- If val  $< 40$ , else 0

$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

### (c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey Google CeLAB Python Program Code
import numpy as np
import matplotlib.pyplot as plt

# Apply averaging image
image = cv2.blur('test.png')
image = matplotlib.pyplot.imshow(image)
print("Thresholded(process)")

# Plotting the transformation mapping
plt.imshow(s, saseFmt="s", lineFmt="b", "b-n")
cv2.thresholds = cv2.blur(image, image)
cv2.thresholds = cv2.threshold([20, 190, 200, 180x=0],
plt.ptttof(sasrre="k=",
plt.xticks([120, 190, 180, 205],
plt.xticks([100, 200, 290, 100],
plt.xtiabel([120, 100, 180, 205],
segmentzn=7)

plt.xlabel(i)
plt.sticks([0, 50, 100, 100, 255])
plt.grid(True, linsStyle="--", alpha=0.5)
plt.show()
```

**Output (Sample)**

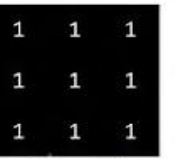
- apply averaging = `cv2.blur((image, [20, 100, 20]), [20, 100, 20])`
- apply thresholding = `cv2.threshold(T=40, ([20, 100, 0, 70, 30], [100, 20, 0, 100])`



Original



Smoothed



Segmented

**Observation:** A multi-stage process with smoothing followed by segmentation improves accuracy by reducing the impact of image noise.

 Computer Vision

 Image Enhancement

 8-bit Grey Levels (0 – 255)

**Code of Conduct:**

*I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

**RUBRICS FOR CALCULATION:**

| <b>Criteria</b>                  | <b>Weightage</b> | <b>Level 4<br/>(Exemplary)</b>                                                           | <b>Level 3<br/>(Proficient)</b>                  | <b>Level 2<br/>(Developing)</b>                   | <b>Level 1<br/>(Beginning)</b>                       |
|----------------------------------|------------------|------------------------------------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------|------------------------------------------------------|
| <b>Problem Understanding</b>     | <b>20%</b>       | <b>Clearly interprets problem, identifies all parameters and requirements accurately</b> | <b>Understands problem with minor gaps</b>       | <b>Partial understanding; misses key elements</b> | <b>Misinterprets or unable to understand problem</b> |
| <b>Method Selection</b>          | <b>20%</b>       | <b>Selects most appropriate method/formula with strong justification</b>                 | <b>Selects correct method with minor errors</b>  | <b>Inappropriate or partially correct method</b>  | <b>Unable to select method</b>                       |
| <b>Solution Process</b>          | <b>25%</b>       | <b>Logical, systematic steps with correct approach throughout</b>                        | <b>Mostly correct steps with minor mistakes</b>  | <b>Disorganized steps; multiple errors</b>        | <b>No clear process</b>                              |
| <b>Accuracy of Calculation</b>   | <b>20%</b>       | <b>All calculations accurate; correct final answer</b>                                   | <b>Minor calculation errors</b>                  | <b>Frequent errors; incorrect final answer</b>    | <b>No valid calculations</b>                         |
| <b>Interpretation of Results</b> | <b>15%</b>       | <b>Clearly interprets results with units and engineering relevance</b>                   | <b>Basic interpretation with minor omissions</b> | <b>Limited or unclear interpretation</b>          | <b>No interpretation</b>                             |

